

Laboratorium 2

Sprawozdanie

Julia Borkowska
Michał Wdowski

9 kwietnia 2021

1 Definicje informacji

1.1 Teoria Shannona - informacja syntaktyczna

1.1.1 Wstęp

Teoria informacji Shannona zajmuje się przepływem informacji ze *źródła* do *odbiorcy*. Źródło emituje *informację*, która jest ciągiem symboli z *alfabetu źródła*. W praktycznych zastosowaniach jest to ciąg skończony, bo każdy przekaz ma swój początek i koniec [4].

1.1.2 Alfabet źródła

Alfabetem źródła nazwiemy zbiór $A_S = \{s_1, \dots, s_n\}$, gdzie s_i reprezentuje pewien **symbol**. Przykładem może być alfabet łaciński z 26 literami.

1.1.3 Informacja

Informacją nazwiemy skończony ciąg elementów z alfabetu źródła razem z ich znaczeniem semantycznym. Przykładem mogą być słowa złożone z liter, które rozumiemy w określony sposób.

1.1.4 Źródło

Źródłem nazwiemy obiekt, który generuje informację. Ogólniej mówiąc, źródło S generuje ciągi $X_1, X_2, X_3, \dots$ zmiennych losowych o rozkładzie $P(X_n = s_i) = p_i, \forall i = [|S|]$. Oznacza to, że na miejscu o indeksie n może się pojawić dany symbol s_i z alfabetu S z określonym prawdopodobieństwem p_i . Jak dla każdego dyskretnego rozkładu prawdopodobieństwa, zachodzą własności: $p_i \geq 0$ oraz $\sum p_i = 1$. W *modelu bez pamięci* rozkład X_n nie zależy od poprzedzających go symboli, jednak później zostaną opisane modele, w których to prawdopodobieństwo od nich zależy.

Przykładem źródła może być kostka sześcienna, która generuje symbole $1, \dots, 6$ z prawdopodobieństwem $\frac{1}{6}$ dla każdego symbolu.

1.2 Informacja semantyczna

1.2.1 Wstęp

Aspekt semantyczny (znaczeniowy) informacji, w odróżnieniu od syntaktycznego (składniowego) skupia się na swego rodzaju wnioskach płynących z informacji. Shannon, tworząc swoją teorię, skupił się na problemie praktycznym i inżynierskim - jak przekazywać informację fizycznymi kanałami, nie skupiając się na jej sensie. U Shannona „Pan Tadeusz” miałby taką samą wartość jak losowe znaki nie układające się w słowa, które mają przypadkiem takie sam rozkład jak słowa języka polskiego. Dlatego potrzebujemy czegoś więcej - ponieważ dla odbiorcy informacja niosąca pewną wiedzę, jak i bezsensowna informacja, są zgoła czymś innym.

1.2.2 Definicja

Artykuł Kolchinsky’ego [3] definiuje semantyczną informację jako „informację, którą *układ fizyczny* posiada na temat otoczenia, która jest wymagana, by układ *podtrzymał swoje istnienie*”. W rozumieniu tej definicji „układem fizycznym” jest pewien obiekt (świadomy bądź nie), który odbiera informację, „podtrzymaniem istnienia” zaś jest osiągnięcie pewnego celu. Przykładem może być komórka, która odbierając bodźce, stara się przetrwać i rozmnożyć. Taka definicja pozwala nam na rozdzielenie informacji na tę *znaczącą* i *nieznaczącą* - znacząca pomaga osiągnąć cel, a nieznacząca to szum, który będzie zignorowany.

1.3 Informacja pragmatyczna

W badaniu informacji ważny jest również jej wpływ na odbiorcę. Informacja jest w stanie zmieniać sposób, w jaki przyszłe informacje zostaną odebrane. W pragmatycznej informacji skupiamy się właśnie na tym wpływie [2]. Kluczowy jest też kontekst - w różnym kontekście ta sama treść może mieć różne konsekwencje dla tego samego odbiorcy. Z tego punktu widzenia, informacje, które prowadzą do tego samego wpływu są sobie równoważne [1]. Przykładem pragmatycznego rozumienia informacji mogą być dwie drogi wskazywane przez różne nawigacje. Ponieważ prowadzą one do tego samego celu, to w pragmatycznym sensie są jednakowe.

1.4 Podsumowanie

Semantyczna i pragmatyczna informacja są pogłębieniem syntaktycznej, potrzebnym by ująć sens i głębię informacji, choć są one trudniejsze do zdefiniowania i objęcia naukowym formalizmem. Z tych powodów w tym sprawozdaniu skupimy się przede wszystkim na informacji syntaktycznej i jej aspektach.

2 Modelowanie informacji

2.1 Model bez pamięci

Najprostszy model bez pamięci jest rozszerzeniem źródła z poprzedniego rozdziału. Możemy założyć, że dla ciągu o N symbolach mamy źródło $S^N = S \times \dots \times S$ - *rozszerzenie źródła stopnia N* - które generuje bloki informacji. Dla każdej pozycji tego ciągu wprowadzamy prawdopodobieństwo $P(X_i = s_i) = p_i$. Oznacza to, że na każdym miejscu każdy symbol ma swoje prawdopodobieństwo wystąpienia, a na każdej pozycji to prawdopodobieństwo może być inne.

2.2 Modele z pamięcią

W poprzedniej sekcji zdefiniowaliśmy model informacji bez pamięci. W rzeczywistych zastosowaniach jest to proste i wygodne, ale często nie oddaje specyfiki informacji. Na przykład litera „z” w języku polskim bardzo często wystąpi po „s” lub „c”, a litery „ą” i „ę” nigdy nie pojawiają się na początku słowa. Z tego powodu konstruuje się modele z pamięcią.

2.2.1 Model warunkowy z pamięcią

Ten model określa prawdopodobieństwo wystąpienia danego symbolu, które zależy od wszystkich poprzedzających go symboli. W takim wypadku mamy:

- alfabet źródła: $A_S = \{s_1, \dots, s_n\}$,
- elementy ciągu informacji - zmienne losowe $X_1, \dots, X_n$
- *konteksty* informacji - zmienne losowe $C_1, \dots, C_n$

Dla każdego X_i znajdujemy rozkład $P(X_i|C_i)$. To, czym będzie wprowadzony kontekst, zależy od danego konkretnego modelu. Może to być kilka poprzedzających symboli, mogą to być wszystkie poprzedzające symbole.

2.2.2 Model Markowa

Model Markowa jest jednym z przykładów modeli warunkowych z pamięcią. Powiemy, że model Markowa *jest rzędu m* , jeśli rozkład prawdopodobieństwa symbolu na danej pozycji zależy od m symboli go poprzedzających.

2.3 Entropia

2.3.1 Informacja własna

Żeby zrozumieć, czym jest entropia, wprowadźmy najpierw pojęcie *informacji własnej* (and. *self-information*). Dla danego wydarzenia $X = x$ jest to wielkość

$$I_X(x) = -\log P_X(x)$$

Jest to miara tego, ile informacji o źródle niesie dana uzyskana wartość. Zauważmy, że im mniejsze prawdopodobieństwo zdarzenia $X = x$, tym większa jest informacja własna. [5]

2.3.2 Entropia

Entropia jest wartością oczekiwaną informacji własnej. W najprostszej wersji, dla pojedynczego źródła, mamy więc wzór

$$H(S) = -\sum_{i=1}^n P(S = s_i) \cdot \log P(S = s_i)$$

Intuicyjnie można to rozumieć jako *średnią informację, jaką niosą poszczególne uzyskane symbole* z A_S .

Warto wspomnieć o podstawie logarytmu. Specjalnie nie została ona zapisana, jako że zależy ona od przyjętej *podstawy*. Jeśli używamy bitów, to zazwyczaj określamy ją jako 2, jeśli używamy cyfr, to jako 10. W ten sposób dobieramy ją do używanych przez nas danych.

3 Kodowanie

Kodowanie polega na zamienianiu postaci informacji na inną wedle ustalonego schematu. Może służyć do różnych celów - w dalszych rozważaniach skupimy się głównie na kompresji informacji, która może zachodzić przy zastosowaniu kodowania. Kompresja polega na zmniejszeniu rozmiaru informacji przy zachowaniu pewnych jej aspektów. Kompresję dzielimy przede wszystkim na stratną i bezstratną. Kompresja bezstratna pozwala na pełne odzyskanie informacji z postaci zakodowanej, natomiast przy kompresji stratnej część informacji jest tracona. Efektywność kompresji wyraża się przez iloraz rozmiaru informacji pierwotnej do zakodowanej.

Pojęciem ściśle związanym z kodowaniem jest redukcja nadmiarowości. Jest to zabieg polegający na oczyszczeniu komunikatu z elementów nie zawierających informacji lub duplikujących informację. Zastosowanie go, podobnie jak kodowanie, pozwala na zmniejszenie rozmiaru przekazywanych informacji bez straty ich samych.

3.1 Kodowanie Huffmana

Kodowanie Huffmana cechuje się prostotą algorytmu, jednak nie jest optymalne pod względem stopnia kompresji danych. Algorytm wyznaczania słów kodowych bazuje na częstotliwości występowania symboli w kodowanym tekście - im częściej dany symbol występuje w badanym tekście, tym krótszy przypisywany kod.

W trakcie wyznaczania kodu jest tworzone drzewo Huffmana, w którym liście zawierają symbole wraz z ich wagami, a inne węzły sumę wag z węzłów-synów. Jest ono niezbędne zarówno do zakodowania informacji, jak i jej odkodowania, dlatego musi zostać przesłane wraz z komunikatem bądź znane odbiorcy wcześniej.

Przy wykorzystaniu kodowania Huffmana zachodzi kompresja bezstratna, co oznacza, że cała oryginalna informacja po zakodowaniu może zostać odzyskana. Powód jest dość oczywisty - każdy symbol z alfabetu wejściowego jest zastępowany unikalnym kodem oraz żaden kod nie jest początkiem innego słowa. Ta druga własność oznacza, że kod ten jest kodem prefiksowym.

Inną ważną cechą tego algorytmu jest jego niedeterministyczność. Niejednoznaczność wyznaczania słów kodowych wynika z braku zasady co do tego, które z łączonych drzew ma być po prawej, a które po lewej stronie oraz który symbol wziąć pierwszy, gdy mają te same wagi. Mimo tego średnia długość kodu pozostaje taka sama.

3.2 Specyfikacja algorytmu Huffmana

3.2.1 Dane wejściowe

Na wejściu algorytmu podawany jest alfabet, czyli zbiór symboli do zakodowania wraz z ich wagami. Wagi są reprezentowane przez prawdopodobieństwo występowania danego symbolu.

3.2.2 Dane wyjściowe

Na wyjściu otrzymywane jest przyporządkowanie kodu w postaci liczby binarnej do symbolu z alfabetu wejściowego.

3.2.3 Kroki algorytmu

1. Utwórz las drzew, w których korzeniach zapisane są symbole z wagami.
2. Weź dwa drzewa, których wagi w korzeniach są najmniejsze i stwórz nowy węzeł, którego synami będą korzenie wybranych drzew.
3. Do korzenia nowego drzewa wpisz wagę, będącą sumą wag jego synów.
4. Jeśli zostało więcej niż jedno drzewo, to wróć do kroku 2.
5. Jeśli zostało dokładnie jedno drzewo, to każdemu symbolowi ze znajdujących się w liściach drzewa przypisz kod na podstawie ścieżki od korzenia do tego liścia - jeśli wybierany jest lewy syn, to do kodu dopisz 0, jeśli prawy - dopisz 1.

3.3 Zależność stopnia kompresji od rodzaju danych

Mimo, że kodowanie Huffmana jest wszechstronnym narzędziem do kompresji danych, to wydajność kompresji w znacznym stopniu zależy od postaci danych. W specyficznych przypadkach można zastosować też niewielkie modyfikacje algorytmu poprawiające stopień kompresji, takie jak grupowanie symboli w wieloznaki. W kolejnych podrozdziałach zostaną przytoczone przykłady obrazujące zależność stopnia kompresji od specyfiki kodowanych danych oraz podstawy do tworzenia drzewa Huffmana.

3.3.1 Kompresja języka naturalnego

Gdy kompresowana informacja jest tekstem w języku naturalnym, można przewidzieć wagi dostępnych w alfabecie znaków na podstawie średnich częstości liter w odpowiednich językach. Dzięki takiemu zabiegowi nie ma potrzeby przesyłania drzewa Huffmana razem z wiadomością, bo obie strony w komunikacji je znają i mogą zdekodować dowolną wiadomość w danym języku. Jako przykład został zbadany tekst literatury pięknej, pierwszy rozdział powieści Fiodora Dostojewskiego "Zbrodnia i kara".

a) Jako pierwszy przeanalizujemy najbardziej realistyczny przypadek tekstu w języku naturalnym, czyli surowy tekst z powieści w angielskim przekładzie.

Oryginalny test zajmował 18 835 bajtów i był zapisany w kodowaniu UTF-8, natomiast ilość znaków w pliku zakodowanym powyższym kodowaniem wygenerowanym przez algorytm Huffmana wynosi 83 706. Ta liczba jest wyrażona w bitach, po konwersji na bajty otrzymujemy ok. 10 463 bajtów, zatem stopień kompresji względem kodowania UTF-8 wynosi ok. 1,8. Entropia źródła policzona z wcześniej przytoczonego wzoru wynosi

ok. 4,47, natomiast średnia długość symbolu wynosi ok. 8,5. Zatem temu kodowaniu daleko do optymalnego pod względem długości kodu - jedną z interpretacji entropii jest długość kodów w optymalnym kodowaniu (w bitach). Jednak dzięki zastosowaniu średnio dłuższych kodów zyskujemy na kompresji całego tekstu, ponieważ wyróżniamy często pojawiające się znaki krótszymi kodami.

Warto przyrzeć się dwóm pierwszym pozycjom w tabeli symboli. Są to znaki nowej linii stosowane w systemach Windows, a na takim właśnie systemie stworzyłam plik tekstowy. Zatem każdy enter jest liczony przez mój algorytm podwójnie, co należałoby dostosować ze względu na niepotrzebne tworzenie nowego symbolu na potrzeby tego samego bitu informacji.

b) Drugi pomiar przeprowadzony został na tekście znormalizowanym - z oryginalnego tekstu zostały usunięte białe znaki i znaki interpunkcyjne, a duże litery zostały zamienione na małe. W ten sposób w tekście zostały tylko litery alfabetu angielskiego.

Symbol	Liczba wystąpień	Prawdopodobieństwo (%)	Kod
a	1156	8,15	1110
b	231	1,63	101100
c	314	2,21	00000
d	589	4,15	11111
e	1706	12,04	011
f	280	1,97	101111
g	332	2,34	00001
h	923	6,51	1001
i	944	6,66	1010
j	13	0,09	0001010111
k	162	1,14	000100
l	586	4,13	11110
m	382	2,69	01010
n	1038	7,32	1100
o	1125	7,93	1101
p	235	1,65	101101
q	15	0,10	000101010
r	750	5,29	0100
s	882	6,22	1000
t	1343	9,47	001
u	390	2,75	01011
v	121	0,85	0001011
w	357	2,51	00011
x	21	0,14	00010100
y	269	1,89	101110
z	5	0,03	0001010110

Entropia źródła wynosi ok. 4,17, średnia długość symbolu - 5,5, zatem pomijając bardzo rzadkie symbole, jak np. niektóre znaki interpunkcyjne zyskaliśmy na długości symboli, a zarazem na wielkości drzewa Huffmana. Kompresja natomiast została zachowana - wynosi ok. 1,9. Warto również zauważyć, że częstości symboli są podobne do średnich częstości liter w języku angielskim, który to fakt wykorzystamy w kolejnym punkcie.

c) Tekst z punktu b) został zakodowany ponownie, tym razem bazując na częstościach liter w języku angielskim. Drzewo Huffmana zostało stworzone na podstawie wartości wyliczonych dla całego współczesnego języka angielskiego.

Symbol	Liczba wystąpień	Prawdopodobieństwo (%)	Kod
a	8200	8,16	1110
b	1500	1,49	110000
c	2800	2,78	01000
d	4300	4,28	11111
e	13000	12,94	100
f	2200	2,19	00100
g	2000	1,99	111100
h	6100	6,07	0110
i	7000	6,97	1011
j	150	0,14	001011010
k	770	0,76	0010111
l	4000	3,98	11001
m	2400	2,38	00110
n	6700	6,67	1010
o	7500	7,46	1101
p	1900	1,89	110001
q	95	0,09	001011001
r	6000	5,97	0101
s	6300	6,27	0111
t	9100	9,06	000
u	2800	2,78	01001
v	980	0,97	001010
w	2400	2,38	00111
x	150	0,14	001011011
y	2000	1,99	111101
z	74	0,07	001011000

Entropia wynosi tyle samo, ponieważ informacja nie zmieniła się. Średnia długość symbolu wyniosła 5,46, zatem niemalże tyle samo, co wcześniej. Podobnie stopień kompresji - rozmiar tekstu z pkt. b) wynosił 59 561 bitów, a tutaj otrzymano 59 657. Z tego wynika, że dla podobnych wielokrotnych transmisji danych, kodowanie może odbywać się w predefiniowanym drzewem Huffmana, które wystarczy przesłać tylko raz. Problemem może wtedy jednak stać się ograniczoność symboli do użycia, jeśli kodowanie ma być

bezstratne. Jeśli natomiast dopuszcza się pominięcie niektórych bardzo rzadkich znaków (jak znaki diakrytyczne innych języków, rzadko używane znaki interpunkcyjne), zysk z kompresji wiadomości jest istotny nawet dla stosunkowo krótkich tekstów.

3.3.2 Kompresja innych tekstów z zastosowaniem symboli blokowych

Poprzednie pomiary dotyczyły jedynie języka naturalnego niespecjalistycznego, a symbole alfabetu były pojedynczymi znakami alfabetu naturalnego. Dowolność doboru symboli w alfabecie daje możliwość innego podejścia do kodowania - symbolami mogą być specyficzne ciągi znaków, np. wyjątkowo często powtarzające się słowa, a symbole nie muszą być tej samej długości. O ile takie podejście nie ma zastosowania w tekstach podobnych do użytego w poprzednim podrozdziale, to daje bardzo dobre wyniki w pewnych specyficznych przypadkach. Dobrym przykładem jest tekst złożony z kodu w pewnym języku programowania, gdzie wyraźnie widać tendencję do powtarzania słów będących częścią języka.

Do pomiarów wykorzystano kod algorytmu napisanego na potrzeby przeprowadzenia kodowania, który został zaimplementowany w języku C#. Tekst został znormalizowany podobnie jak w poprzednim przykładzie z fragmentem powieści - usunięto enter, a pomiędzy każdym słowem traktowanym jako symbol stoi jedna spacja.

Tekst podzielony na pojedyncze znaki ma entropię ok. 4,22, średnia długość symbolu wynosi ok. 8,25, zatem drzewo Huffmana jest dość duże. Stopień kompresji wyniósł natomiast ok. 1,65, zatem dość podobnie, jak w języku naturalnym, a na obniżenie tego parametru wpłynęły bardzo różnorodne znaki interpunkcyjne obok liter, a same litery są rozróżniane na małe i wielkie.

Tekst podzielony na symbole wieloznakowe na inną entropię ze względu na inny alfabet, którym się posługuje - wyniosła ona ok. 6,02. Średnia długość symbolu wzrosła nieznacznie do 8,84. Co jednak najważniejsze, stopień kompresji wzrósł do 6,34. Jest to znacznie więcej niż w innych badanych przypadkach. Wynika z tego, że sposób dzielenia kodowanego tekstu ma wielkie znaczenie w stopniu kompresji danych. Takie podejście wymaga jednak wstępnej analizy kodowanego tekstu i nie jest uniwersalne.

4 Eksperymentalna redukcja nadmiarowości

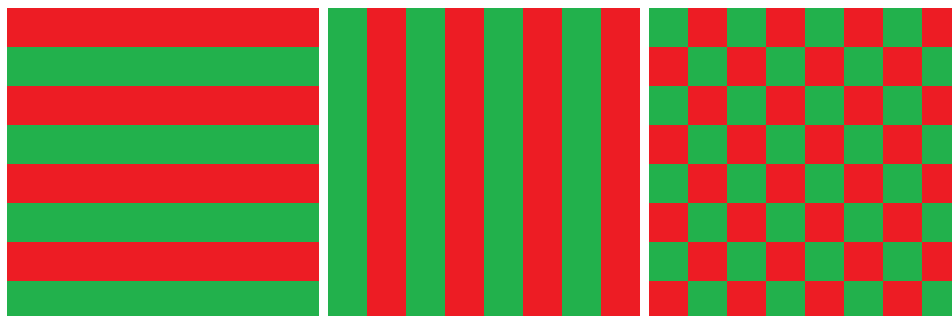
4.1 Wstęp

W tej sekcji przyjrzymy się bezstratnym formatom grafiki rastrowej - PNG, TIFF i BMP. Porównamy objętości kilku różnych obrazów w każdym z formatów i opiszemy, w jaki sposób są one zapisywane. następnie przyjrzymy się narzędziom poświęconym teorii informacji - głównie syntaktycznej wg. Shannona. Ponieważ zostały nam zaprezentowane biblioteki języka MATLAB, to my spróbujemy znaleźć adekwatne alternatywy w języku Python.

4.2 Porównanie bezstratnych formatów grafiki rastrowej

4.2.1 Dane

Będziemy zajmować się trzema obrazkami zapisanymi w różnych formatach grafiki rastrowej. Wszystkie trzy mają dwa kolory, a ich wymiary to 64×64 pikseli. Jako „ten sam obrazek w różnych formatach” będziemy rozumieć coś, co *wygląda tak samo* - a zatem w sensie semantycznym i pragmatycznym dla ludzkiego obserwatora jest to ta sama informacja.



Obrazek 1: horizontal

Obrazek 2: vertical

Obrazek 3: checkerboard

4.2.2 Wyniki

W poniższej tabeli zostały przedstawione objętości obrazków w poszczególnych formatach. Jednostką objętości na dysku są bajty. Do stworzenia plików PNG i BMP użyto programu MSPaint, a do TIFF - programu GIMP.

Format	horizontal	vertical	checkerboard
PNG	233	279	289
BMP	12 342	12 342	12 342
TIFF	12 537	12 535	16 651

Zanim przejdziemy do komentarza wyników, przyjrzyjmy się jeszcze jednej sprawie. Policzmy, ile bajtów potrzebujemy, by zapisać taki obrazek bez kompresji. Mamy 64 pikseli wysokości i szerokości, a każdy piksel przechowuje informacje o zakresie ze skali RGB - między 0 a 255 - a więc po jednym bajcie. Może się pojawić też czwarta skala - przezroczystości. Mamy zatem

$$64 \cdot 64 \cdot 3 = 12288 \text{ bądź } 64 \cdot 64 \cdot 4 = 16384,$$

w zależności od obecności skali przezroczystości. Oczywiście żaden format, przy braku kompresji, nie osiągnie takich wyników, ponieważ trzeba założyć, że pliki zachowują też metadane o samym formacie czy chociażby wymiarach obrazka.

4.2.3 Komentarz do wyników

Najbardziej rzuca się w oczy różnica rzędów wielkości między PNG a pozostałymi formatami. Sugeruje to, że za tym formatem stoi jakiś błyskotliwy sposób kompresji. W PNG ciekawa jest też różnica między poszczególnymi obrazkami - paski poziome kompresują się lepiej niż pionowe.

W formacie BMP wszystkie obrazki mają tę samą objętość. Bardzo możliwe, że żadna kompresja tam nie zachodzi, tylko poszczególne piksele są zapisywane po kolei - takimi jakie są. Świadczy za tym bliskość wynikowej objętości do policzonej teoretycznej objętości bez kompresji dla formatów bez skali przezroczystości. Format TIFF sprawił się gorzej od mapy bitowej, ale to może być kwestia obecności skali przezroczystości w tym formacie. Być może wymiary obrazków były za małe, by TIFF pokazał swoje możliwości - w końcu szachownica była blisko liczby bajtów. Warto zwrócić uwagę na różnice dwóch bajtów między paskami poziomymi i pionowymi. Być może gdzieś w tym formacie jest podejmowana decyzja, czy kompresujemy idąc poziomo czy pionowo, a może jest tam jeszcze inny sposób kompresji.

W następnych sekcjach postaramy się zgłębić te trzy formaty.

4.2.4 PNG

PNG (ang. *Portable Network Graphics*) tworzy swoje pliki w dwóch etapach. Pierwszym jest filtrowanie - dla każdego piksela, na podstawie pikseli z lewej, z góry i z górnego-lewego po przekątnej, liczona jest wartość. Tych filtrów jest 5 - różnice między nimi, średnia, albo predykcja Paetha (pewna liniowa funkcja tych 3 pikseli). Dla każdej poziomej linii ustalany jest taki filtr, który minimalizuje liczbę różnych wartości w danej linii. Następnie na danych po filtrze stosowana jest kompresja algorytmem DEFLATE, który jest kombinacją kodowania Huffmana z algorytmem LZ77. Kod Huffmana został przedstawiony wcześniej w tekście. LZ77 z kolei szuka, czy dany ciąg symboli pojawił się wcześniej. Jeśli tak się stało, to można go zastąpić informacją, że X symboli temu powtórzył się Y symboli. W kontekście obrazków ta metoda okazuje się niezwykle skuteczna.[6]

4.2.5 BMP

BMP (ang. *Bitmap*) okazuje się być niezbyt ciekawym formatem. Przechowuje on meta-dane i wartości pikseli ze skali RGB. Żadna kompresja tak naprawdę nie zachodzi.

4.2.6 TIFF

TIFF (ang. *Tag Image File Format*) jest należącym do Adobe formatem grafiki rastrowej. Oferuje on wiele wybieranych opcji dla danego pliku. Można dobrać skalę kolorów czy sposoby kompresji. Bazowym algorytmem kompresji jest *PackBits*. Koduje on powtarzające się symbole jako „ X powtórzył się Y razy” bądź mówi, że następne symbole są przedstawione w dosłowny sposób. W rozszerzeniach dostępne są inne sposoby, np. algorytm LZW. Znajduje on coraz dłuższe powtarzające się serie symboli, i zastępuje odpowiednim kodowanym symbolem.

4.3 Narzędzia do teorii informacji

4.3.1 Python - dit

Biblioteka **dit** pozwala, po wprowadzeniu danych dla rozkładu, znajdować wiele własności tego rozkładu, które pochodzą z zakresu teorii informacji. Opiera się na budowaniu rozkładów prawdopodobieństwa, dlatego np. eksperymentalne liczenie entropii obrazka wymagałoby najpierw zakodowania obrazka i konwersji do rozkładu.

```
import dit
die_throws = {'1': 1/6, '2': 1/6, '3': 1/6, '4': 1/6, '5': 1/6, '6': 1/6}
die_distribution = dit.Distribution(die_throws)
die_distribution
```

Class:	Distribution	x[0]	p(x)
Alphabet:	('1', '2', '3', '4', '5', '6') for all rvs	1	0.16666666666666666
Base:	linear	2	0.16666666666666666
Outcome Class:	str	3	0.16666666666666666
Outcome Length:	1	4	0.16666666666666666
		5	0.16666666666666666
		6	0.16666666666666666

Możemy wypisać własności rozkładu w przystępnej formie w notatniku jupyter bądź w konsoli interpretera Python. Spróbujmy teraz wykonać taki rozkład dla obrazka z zakodowanymi pikselami:

```
# import bibliotek
import numpy as np
from PIL import Image

# import obrazka
im_frame = Image.open('../images/checkerboard.png')
np_frame = np.array(im_frame.getdata())

# funkcja wyciągająca elementy i ich prawdopodobieństwo wystąpienia z macierzy
def rozklad(array):
    elementy, zliczenia = np.unique(array, return_counts=True, axis=0)
    prawdopodobienstwa = np.divide(zliczenia, array.shape[0])
    return (elementy, prawdopodobienstwa)

# wyliczenie rozkładu dla naszego obrazka
elementy, prawdopodobienstwa = rozklad(np_frame)
elementy_indeksy = []
```

```

# kodowanie różnych elementów jako liczby
for i in range(len(elementy)):
    elementy_indeksy.append(str(i))

# stworzenie słownika par element - prawdopodobieństwo
dictionary = {}
for A, B in zip(elementy_indeksy, prawdopodobienstwa):
    dictionary[A] = B
dictionary

# konwersja słownika do rozkładu dit
dit.Distribution(dictionary)

```

Class:	Distribution	x[0]	p(x)
Alphabet:	('0', '1') for all rvs	0	0.5
Base:	linear	1	0.5
Outcome Class:	str		
Outcome Lenght:	1		

Zgadza się to z naszą wiedzą, że obrazek szachownicy dwukolorowej ma dwie kodowane wartości. Największą trudnością w doświadczeniu z tą biblioteką okazuje się konwersja naszej informacji do rozkładu prawdopodobieństwa. Dokumentacja dit jest dobrym wprowadzeniem, ale słabą dokumentacją. Nie można znaleźć listy wszystkich metod i ich opisów zachowań. Z tego powodu konwersja z informacji do rozkładu prawdopodobieństwa jest robiona samodzielnie.

Literatura

- [1] Andrew Frank. *Foundations of Geographic Information Science*. Taylor Francis, 2003.
- [2] D. Gernert. *Pragmatic Information: Historical Exposition and General Overview*. 2006.
- [3] Kolchinsky A. i Wolpert D. H. *Semantic information, autonomous agency and non-equilibrium statistical physics*. Royal Society, 2018.
- [4] J. M. Jones, Gareth A. i Jones. *Information and Coding Theory*. Springer-Verlag, 2000.
- [5] Aerin Kim. *The intuition behind Shannon's Entropy*. Towards Data Science, 2018.
- [6] Colt McAnlis. *How PNG Works*. medium.com, 2016.

5 Inne źródła

Tekst powieści ze strony Projektu Gutenberg - <https://www.gutenberg.org/>, częstości liter w języku angielskim - Wikipedia - https://en.wikipedia.org/wiki/Letter_frequency.

6 Podział pracy

1. Definicje informacji - Michał Wdowski
2. Modelowanie informacji - Michał Wdowski
3. Kodowanie - Julia Borkowska
4. Eksperymentalna redukcja nadmiarowości - Michał Wdowski